LAB ASSIGNMENT 3
NUMBA ACCELERATION

OBJECTIVES

Upon completion of this lab, you will be able to:

1. Understand how GPU hardware accelerates different implementations of algorithms and assess the performance of the accelerated code.
2. Get familiar with a Python code acceleration tool - numba.
3. Make your Python code faster with minor modifications.
4. Compare how computing can be accelerated in CPU and GPU.

LAB REPORT

1. You can create one or more Jupyter notebooks during the following tasks. The combination of these exported as PDF will be your lab report.
2. Use proper formatting and markdown between tasks or different code blocks. You can look for documentation online, for example: https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet
3. Your code should be commented, and all the outputs should be visible in the notebook. You should also submit the notebook itself (right-click and download on the left menu) so that we can run all the code again.
4. Grade will be 50% thoroughness of results, 20% answers to the different questions, 20% quality of images/plots and 10% readability of code and report.

PRELIMINARY WORK

If you have a Nvidia GPU in your laptop, you can complete the lab at home.

If no NVIDIA GPU is available, access must be done through the laboratory. Go to Tierenkatu 8, connect to the TIARIS network, then open the GPU server at http://192.168.50.78:8000/.

Several python modules may be required during the assignment. If importing a module fails, it is likely not installed. Install it using:

```
!pip install module_name
```

For example:

```
!pip install numba numpy
```

If the import still fails after installation, restart the kernel and import again. To restart the kernel, click *"Kernel"* in the top menu and select *"Restart Kernel…"*.

To avoid losing progress in case of a server issue, download your notebook regularly while working: In the notebook, click *"File"* in the top menu. Select *"Download"*. This will save the file to your local machine. Repeat this periodically during the assignment, especially after completing important sections.

TASK 1 – ACCELERATION IN CPU WITH NUMBA

In this task, you will accelerate some sample codes. The main code is for calculating the value of π using a monte carlo method.

1. Import the necessary packages

```
import numba as nb
import numpy as np
```

2. The main code for calculating $\pi$is as follows:

```
def monte_carlo_pi(nsamples):
    acc = 0
    for i in nb.prange(nsamples):
        x = np.random.random()
        y = np.random.random()
        if (x**2 + y**2) < 1.0:
            acc += 1
    return 4.0 * acc / nsamples
```

3. Test the ability of *numba* to accelerate this function:
   1) Run normally

```
monte_carlo_pi(1000)
```

   2) Run with numba accelerated in CPU (run it twice)

```
monte_carlo_pi_jit = nb.njit()(monte_carlo_pi)
monte_carlo_pi_jit(1000)
```

   3) Run with multi-threads parallely in CPU

```
monte_carlo_pi_jit_par = nb.njit(parallel=True)(monte_carlo_pi)
monte_carlo_pi_jit_par(1000)
```

In your report, answer the following questions
(briefly and with your own words):

(i) Why the first time you run the code with numba it is slower than the baseline python implementation?

(ii) Draw a line chart of the cost time changing relative to the variable *nsample* with range from 100 to 100000 and having **logarithmic axes**.

(iii) What does the param *nopython* mean in *numba.jit()* function?

TASK 2 – ACCELERATING IN CPU AND GPU WITH VECTORIZE

Now we will use the *@vectorize* decorator to accelerate both in CPU and GPU some simple element-wise operations on arrays.

In class we saw how to accelerate a basic numpy ufunc: the element-wise addition of arrays. Choose **two other ufuncs** (a simple one like multiplication or division, and some more complex one like a log).

1. Write three versions of the function: python version, numba vectorize, numba vectorize with parallel execution in CPU (set target="parallel"), and vectorize with target GPU (set "target="cuda").

2. Time the function for array sizes varying several orders of magnitudes (get as big as you can without having to wait too long for execution or running out of memory on the GPU side).

3. Draw a figure with logarithmic axes showing the latency based on the input size.

4. Draw the same figure again but now use 64-bit precision (try with both integer and floating point, you may choose how many graphs to use to report the results). Comment in your report on the differences in performance for the different implementations.

5. Vectorize is a powerful decorator that we can also use with more complex functions. Now implement and accelerate a function that calculates the distances between two sets of points in polar coordinates (given in four numpy arrays).
6.
   (1) Define your inputs as follows:

```
n = 1000000
rho1 = np.random.uniform(0.5, 1.5, size=n).astype(np.float32)
theta1 = np.random.uniform(-np.pi, np.pi, size=n).astype(np.float32)
rho2 = np.random.uniform(0.5, 1.5, size=n).astype(np.float32)
theta2 = np.random.uniform(-np.pi, np.pi, size=n).astype(np.float32)
```

   (2) We can easily calculate the distance in cartesian coordinates using numpy ufuncs. To do the conversion, however, we will use a *device function*, a function that will be running only on the GPU and can be called within the kernels that *@vectorize* spawns:

```
@cuda.jit(device=True)
def polar_to_cartesian(rho, theta):
x = rho * math.cos(theta)
y = rho * math.sin(theta)
return x, y
```

   (3) Compare the performance in CPU and GPU.

TASK 3 – CUSTOM CUDA KERNELS

We have seen in the lecture a basic CUDA kernel implementation with numba that calculates the square root. We will iterate over this one and do some further analysis on the best way of defining the grid parameters.

1. Write the *sqrt()* kernel implementation from the slides on a Jupyter notebook.

2. Compare the performance for various input sizes of a baseline python version, numba CPU accelerated, vectorized versions (parallel and GPU) and also with the new custom kernel. Comment on your results.

3. Now we will be modifying the number of blocks per grid and the number of threads per block. Remember that since we are only using the kernel for a single element, we need the total number of threads to be the same as the input size. Try at least 4 different grid setups and choose the best way of plotting your results (e.g., bar graphs or a 2D heatmap, among others).

4. What happens when the input size grows considerably? What type of error do you get? Any idea on how this could be solved?

5. From the total execution time, how much time goes to copying the inputs and outputs?

6. Take the same kernel but implement it so that each thread can process multiple input elements. Now again study the performance when you change the grid size and the number of threads, but such that the total number of threads is at maximum half of the input size.

TASK 4 – MULTIDIMENSIONAL KERNELS

Now write a function that, given a matrix, computes a function element-wise using a 2D cuda grid (without flattening the matrix to a 1D array).

1. Define a device function that performs the single-element operation.
2. Define the cuda kernel which calls the previous device function.
3. How does the performance change when the matrix size changes? Compare a baseline CPU version (without numba) and GPU versions.
4. Compare a simple and a fairly complicated operation. Does the timing change significantly for larger inputs?
5. (OPTIONAL) Compare the performance to one or more of the numba optimizations we have previously seen
6. (OPTIONAL) Implement the multiplication of two matrices with a cuda kernel.

TASK 5 – BASIC IMAGE PROCESSING

Finally, extend the previous task to perform some basic image processing:

1. The input is now an image but each 2D element actually has 3 dimensions/colors.
2. Implement a (customized) BW filter (weight-average of the three channels). The output has only one dimension per pixel. Be creative with your weights (i.e., choose which color is more "important").
3. **(Optional for Grade 5)** Implement a gaussian blur filter.